

Java is a programming language.

JDBC is used to connect Java to databases.

Spring Boot is a framework.

RAG stands for Retrieval Augmented Generation.

FAISS is a vector database library.

HashMap

A **HashMap** stores **key-value pairs**, which are great when you want to store values and find them by a key (like a name or ID):

Example

```
// Import the HashMap class
import java.util.HashMap;

public class Main {
    public static void main(String[] args) {
        // Create a HashMap object called capitalCities
        HashMap<String, String> capitalCities = new HashMap<String,
String>();

        // Add keys and values (Country, City)
        capitalCities.put("England", "London");
        capitalCities.put("Germany", "Berlin");
        capitalCities.put("Norway", "Oslo");
        capitalCities.put("USA", "Washington DC");
        System.out.println(capitalCities);
    }
}
```

Data Structures Overview

Data Structure	Stores	Keeps Order?	Allows Duplicates?	Best For
ArrayList	Ordered elements	Yes	Yes	Accessing elements by index
HashSet	Unique elements	No	No	Avoiding duplicates, fast checks
HashMap	Key-value pairs	No	Yes (keys are unique)	Fast lookup by key

Iterators

When learning about data structures, you will often hear about iterators too.

An iterator is a way to loop through elements in a data structure.

It is called an "iterator" because "iterating" is the technical term for looping.

Example

Using an Iterator with ArrayList:

```
import java.util.ArrayList;
```

```
import java.util.Iterator;

public class Main {
    public static void main(String[] args) {
        // Create an ArrayList of Strings
        ArrayList<String> cars = new ArrayList<String>();
        cars.add("Volvo");
        cars.add("BMW");
        cars.add("Ford");
        cars.add("Mazda");

        // Get an iterator for the ArrayList
        Iterator<String> it = cars.iterator();

        // Iterate through the list using the iterator
        while(it.hasNext()) {
            System.out.println(it.next());
        }
    }
}
```

Next, let's take a closer look at each data structure in more detail.

Advanced Java Concepts

Exception Handling

The Java Exception Hierarchy is a structured system where all exception classes are subtypes of the Throwable class. Key components include the 'finally' block, which is used to execute important code such as closing connections, and the 'throw' keyword, used to explicitly throw an exception. Exception propagation occurs when an exception is not caught in the current method and is passed up the call stack to the caller.

Collections Framework

The Java Collections Framework provides a unified architecture for representing and manipulating collections. It includes core interfaces such as List (ordered elements), Set (unique elements), Queue (FIFO/LIFO structures), and Map (key-value pairs). Generics are essential for ensuring type safety within these collections.

Iterators are used to loop through or "iterate" over elements in a data structure. Additionally, advanced features like the Streams API allow for functional-style operations on elements, while Hashing algorithms power efficient storage and retrieval in structures like HashMap.